Lab 09 — RBAC, Secrets, ServiceAccounts & NetworkPolicies

ID: K8S-LAB-09 · **Category:** Enterprise · **Level:** Enterprise · **Duration:** 60 min

Focus: RBAC, Roles, RoleBindings, Secrets, ServiceAccounts, NetworkPolicy

Objective

Lock down a namespace with least-privilege RBAC, mount a Secret safely, and apply a default-deny NetworkPolicy.

Concepts Covered

• Role vs ClusterRole

• ServiceAccount tokens

• Why Secrets aren't really 'secret' by default

• Default-deny network posture

Prerequisites

• A cluster from Lab 1

Architecture

RBAC is just three objects: Role describes *what*, RoleBinding describes *who*, ServiceAccount provides the identity.

```
Subject (User/SA/Group)  --(RoleBinding)-->  Role  --(rules)-->  Resources
```

Lab Steps

01. Create a namespace and a junior SRE ServiceAccount

Identity for our least-privilege actor.

```
kubectl create namespace ops
kubectl -n ops create serviceaccount sre-junior
```

Expected: namespace/ops and serviceaccount/sre-junior created.

Internals: Every namespace also gets a default ServiceAccount automatically.

02. Define a read-only Role

Lets sre-junior read pods/logs/events in `ops` only.

```
cat <<EOF | kubectl -n ops apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: read-only}
rules:
- apiGroups: [""]
  resources: ["pods","pods/log","events"]
  verbs: ["get","list","watch"]
EOF
kubectl -n ops create rolebinding sre-junior-read --role=read-only --serviceaccount=ops:sre-junior
```

Expected: role and rolebinding created.

Internals: RBAC is additive and namespaced unless a ClusterRole/ClusterRoleBinding is used.

03. Test as the ServiceAccount

auth can-i is the fastest way to verify RBAC.

```
kubectl -n ops auth can-i list pods --as=system:serviceaccount:ops:sre-junior
kubectl -n ops auth can-i delete pods --as=system:serviceaccount:ops:sre-junior
```

Expected: yes / no.

Internals: API server simulates the RBAC check with the requested identity.

04. Create and mount a Secret

Secret is injected via env. Logs show the value — proving the mount, but also why Secrets need RBAC + encryption-at-rest.

```
kubectl -n ops create secret generic db --from-literal=password=s3cret
kubectl -n ops run app --image=busybox --restart=Never -- sh -c 'echo $DB_PASS; sleep 3600' --overrides='{
  "spec": {
    "containers": [{
      "name":"app","image":"busybox",
      "command":["sh","-c","echo $DB_PASS; sleep 3600"],
      "env":[{"name":"DB_PASS","valueFrom":{"secretKeyRef":{"name":"db","key":"password"}}}]
    }]
  }
}'
kubectl -n ops logs app
```

Expected: `s3cret`

Internals: Secrets are base64-encoded, not encrypted, by default. Enable EncryptionConfiguration on the API server for real protection.

05. Apply default-deny ingress and re-allow only what's needed

All inbound traffic to Pods in `ops` is now blocked unless explicitly allowed.

```
cat <<EOF | kubectl -n ops apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-ingress}
spec:
  podSelector: {}
  policyTypes: [Ingress]
EOF
```

Expected: networkpolicy.networking.k8s.io/default-deny-ingress created.

Internals: Requires a CNI that enforces NetworkPolicy (Calico/Cilium). Flannel needs the kube-router or canal variant.

Common Errors

NetworkPolicy has no effect — Cause: CNI doesn't enforce them (e.g. vanilla Flannel) · Fix: Use Calico/Cilium or add kube-router

Forbidden errors even after RoleBinding — Cause: Wrong namespace or wrong subject kind · Fix: auth can-i --as=... pinpoints it

Troubleshooting

• `kubectl auth can-i --list -n ops --as=...` enumerates effective rights

• `kubectl get rolebinding,clusterrolebinding -A | grep <sa>`

Enterprise Relevance

RBAC + NetworkPolicy + encrypted Secrets is the minimum bar for a compliant cluster (SOC2, ISO27001, PCI).

Validation

• sre-junior can read but not write

• Default-deny NetworkPolicy is active in `ops`

Related Incidents & Guide Sections

Incidents: K8S-INC-3005

• RBAC & PodSecurityAdmission

Questions

Beginner

Q. What is a ServiceAccount for?

A. An identity for in-cluster workloads to authenticate to the API.

Intermediate

Q. Difference between Role and ClusterRole?

A. Role is namespaced; ClusterRole is cluster-wide or used for non-namespaced resources.

Advanced

Q. Why do default Secrets not satisfy compliance?

A. They're only base64 in etcd unless EncryptionConfiguration is enabled.

Scenario

Q. A pod is exfiltrating data. How do you block egress immediately?

A. NetworkPolicy with Egress type and an empty allow list, scoped to the pod's labels.

Viva

Q. Walk through `kubectl auth can-i`.

A. Sends a SubjectAccessReview to the API server with the supplied user; returns the RBAC decision.